

Today:

- Syllabus/Outline
- Tools/Environments
- HWO
- Bash (Unix Shell)

This lecture material is adapted from <https://swcarpentry.github.io/shell-novice/index.html> . The data used for the examples below is available from <https://swcarpentry.github.io/shell-novice/data/shell-lesson-data.zip>

The overall goal of this course is to enable you to generate reproducible results from scientific computing software tools and to be able to generate publication quality figures and tables from data generated in this way.

Along the way, we'll consider:

- What libraries and modules are available to help you in this process?
- How to write correct code
- Understanding computational cost and efficiency, and how to make code "run faster"

A place to start: the Unix shell

Most of what we do on computers is through a GUI (graphical user interface), where we use a mouse to navigate menus and select commands to run.

This is fine for writing an email, or sorting a spreadsheet, but breaks down for more complicated tasks.

A lot of scientific computing happens at the "command line", using the keyboard to enter text commands:

- submit jobs to supercomputers
- work with data spread across many files
- manage and compile software projects with complex dependencies

If you can't use the command line, your ability "do scientific computing" can be limited!

Many variants on Unix shells

- first ones date from 1970's
- csh, bash, tcsh, ksh, zsh, ...

If you already know and like a particular shell, just continue to use it.

- I use csh on my laptop, have for many years
- Default shell on modern machines is usually bash, so I'll use this for examples

Notes for later:

- customized command to log on
- No password, use "ssh keys"

Left with prompt:

In []:

```
scottmac@kaczmarz:~$
```

What can we do?

```
scottmac@kaczmarz:~$ ls
adaptive.patch      github.token        monolithicmg_2018  rtsolver.py
Amin_NS30.py        heat_figure1.png    more.tgz            rtsolver.py~
arc42_nosmooth_IG.log heat_figure2.png    Music               Scott_CMSC6950
August_2023          heat_figure32.png  myfigs.pdf          Sept_2022
CMSC6950             heat_figure.png     new-0.tgz           sing_perturb
CMSC6950_Fall2024    heat_figure_stronger.png new-1.tgz           smectic1.tgz
coarse_grid_selection_sa heat_figure_tol.png new-smectic.tgz     smectic2.tgz
convergence.pdf      heat_movie_data.tgz nohup.out           snap
defcon               hermite_diff        non_aligned.py      STREAM
Desktop              Irksome              November_2023       STREAM-python
Documents             Irksome              numpebook           Templates
Downloads             January_2023         numpebook-homeworks test_HW4.py
Dropbox               January_2024         output1.mp4         time_dep_stokes.py
examples.desktop      latex                output.mp4          tmp
firedrake             March_2024           perf.sh             umr_sil_10ppw_16cubeUM.py
firedrake-April2025   Marie-Curie-Proposal perf.sh~            umr_sil_10ppw_16cubeUM.py~
firedrake_codes       math6204_HW         petsc               unitcircle.msh
firedrake-configure   math6210_HW         Pictures            Videos
firedrake-install     May_2024             poisson_neumann_id.py viz.tgz
firedrake-install.log MMS-directfail.py   Public
firedrake-March2025   MMS-directfail.py~ pyamq
footing.tgz           mms.tgz              razan
```

Don't be afraid to make mistakes...

```
scottmac@kaczmarz:~$ dsfjjhdfsjkh
dsfjjhdfsjkh: command not found
```

How do we navigate?

- `cd` or `chdir` command to change directories

```
scottmac@kaczmarz:~$ cd CMSC6950
scottmac@kaczmarz:~/CMSC6950$ ls
shell-lesson-data  shell-lesson-data.zip
scottmac@kaczmarz:~/CMSC6950$ cd shell-lesson-data/
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$
```

Magic: tab-completion, type first few letters of a command/directory/filename and hit tab. Should autocomplete, unless typo or ambiguity (double-tab to show possible completions)

Lots of files in demo directory

Useful commands:

- `pwd` to print working directory
- `ls -F` "classifies" entries in directory listing
- `clear` or `ctrl-l` clears display

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ pwd
/home/scottmac/CMSC6950/shell-lesson-data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ ls -F
exercise-data/  north-pacific-gyre/
```

Magic:

- use arrows at command line to scroll through your history
- use `!!` to repeat last command
- use `!C` to repeat last command that started with a c

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ ls -F
exercise-data/  north-pacific-gyre/
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ !!
ls -F
exercise-data/  north-pacific-gyre/
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ clear
```

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ !!
ls -F
exercise-data/  north-pacific-gyre/
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$
```

Note that, as a command, `ls` accepts many options. 3 ways to get help with understanding these

- `ls --help`
- `man ls`
- google (or ChatGPT/grok)

Some useful options:

- `ls -l` shows permissions, sizes, dates, and names of files
- `ls -t` sorts by last modified time
- `ls -r` reverses order
- `ls -ltr` does all 3

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ man ls
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ ls -l
total 8
drwxr-xr-x  6 scottmac scottmac 4096 Mar 12  2023 exercise-data
drwxr-xr-x  6 scottmac scottmac 4096 Mar 12  2023 north-pacific-gyre
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ ls -t
exercise-data  north-pacific-gyre
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ ls -ltr
total 8
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 north-pacific-gyre
drwxr-xr-x  6 scottmac scottmac 4096 Mar 12  2023 exercise-data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ cd exercise-data/
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data$ ls -ltr
total 20
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 writing
-rw-r--r--  1 scottmac scottmac  13 Mar 12  2023 numbers.txt
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 creatures
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 animal-counts
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 alkanes
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data$ cd ..
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$
```

Two ways to specify paths in unix:

- By default, all paths are relative to current working directory. Use `..` to go back to parent directory, and use `/` to separate directories

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ cd ../../
scottmac@kaczmarz:~$ cd CMSC6950/shell-lesson-data/exercise-data/
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data$
```

We can also use *absolute* paths, starting with `/`

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data$ ls /
bin          etc          lib32        media        run          swapfile    var
bin.usr-is-merged home         lib64        mnt          sbin         sys          vmlinuz
boot         initrd.img  lib.usr-is-merged opt          sbin.usr-is-merged target       vmlinuz.old
cdrom        initrd.img.old libx32       proc         snap         tmp
dev          lib         lost+found  root         srv          usr
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data$ cd /home/scottmac/CMSC6950/shell-lesson-data/
```

Magic:

- the path `~` is short for `/home/username`

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ cd ~/CMSC6950
```

Magic:

- `cd -` brings you back to previous directory
- `pushd / popd` allow you to create "stacks" of directories and move through them. `dirs` shows stack.

```
scottmac@kaczmarz:~/CMSC6950$ cd -
/home/scottmac/CMSC6950/shell-lesson-data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ pushd ~/CMSC6950_Fall2024/git/
~/CMSC6950_Fall2024/git ~/CMSC6950/shell-lesson-data
scottmac@kaczmarz:~/CMSC6950_Fall2024/git$ pushd ~/CMSC6950/shell-lesson-data/exercise-data/
alkanes/ animal-counts/ creatures/ writing/
scottmac@kaczmarz:~/CMSC6950_Fall2024/git$ pushd ~/CMSC6950/shell-lesson-data/exercise-data/alkanes/
~/CMSC6950/shell-lesson-data/exercise-data/alkanes ~/CMSC6950_Fall2024/git ~/CMSC6950/shell-lesson-data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/alkanes$ dirs
~/CMSC6950/shell-lesson-data/exercise-data/alkanes ~/CMSC6950_Fall2024/git ~/CMSC6950/shell-lesson-data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/alkanes$ popd
~/CMSC6950_Fall2024/git ~/CMSC6950/shell-lesson-data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ popd
~/CMSC6950/shell-lesson-data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$
```

Two good uses of these:

- Interruptions, where you need to go do something else and then come back.
- Copying code/files from somewhere else

Modern terminals also allow tabs

Can "break up" shell commands as

```
[prompt] [command] [options] [arguments]

scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ ls -ltr exercise-data/
total 20
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 writing
-rw-r--r--  1 scottmac scottmac  13 Mar 12  2023 numbers.txt
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 creatures
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 animal-counts
drwxr-xr-x  2 scottmac scottmac 4096 Mar 12  2023 alkanes
```

We can create directories with `mkdir`

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data$ cd exercise-data/writing/
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ ls
haiku.txt  LittleWomen.txt
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ mkdir thesis
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ ls -ltr thesis/
total 0
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$
```

`mkdir -p` creates parent directories as needed

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ mkdir -p ../project/data
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ mkdir -p ../project/results
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ ls -R ..
..
alkanes  animal-counts  creatures  numbers.txt  project
..
alkanes:
cubane.pdb  ethane.pdb  methane.pdb  octane.pdb  pentane.pdb  propane.pdb
..
animal-counts:
animals.csv
..
creatures:
basilisk.dat  minotaur.dat  unicorn.dat
..
project:
data  results
..
project/data:
..
project/results:
..
writing:
haiku.txt  LittleWomen.txt  thesis
..
writing/thesis:
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$
```

We can create a blank files either with the `touch` command (to creates an empty file) or by opening one in an editor

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ touch file1.txt
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ ls
file1.txt  haiku.txt  LittleWomen.txt  thesis
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ nano file2.txt
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ ls -ltr
total 1044
-rw-r--r--  1 scottmac scottmac 1053440 Mar 12  2023 LittleWomen.txt
-rw-r--r--  1 scottmac scottmac  218 Mar 12  2023 haiku.txt
drwxrwxr-x  2 scottmac scottmac  4096 Sep  9 11:35 thesis
-rw-rw-r--  1 scottmac scottmac  0 Sep  9 11:37 file1.txt
-rw-rw-r--  1 scottmac scottmac  17 Sep  9 11:38 file2.txt
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$
```

We can delete files using `rm`

```
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ rm file1.txt
scottmac@kaczmarz:~/CMSC6950/shell-lesson-data/exercise-data/writing$ ls -ltr
total 1044
-rw-r--r--  1 scottmac scottmac 1053440 Mar 12  2023 LittleWomen.txt
-rw-r--r--  1 scottmac scottmac  218 Mar 12  2023 haiku.txt
drwxrwxr-x  2 scottmac scottmac  4096 Sep  9 11:35 thesis
-rw-rw-r--  1 scottmac scottmac  17 Sep  9 11:38 file2.txt
```

Deleting is permanent! No "trash"

Notes:

- `nano` is a very basic editor. It works, but you might find it valuable to learn a more fully featured text editor that you can run at the command line. Commonly, we use `vim` or `emacs`
- use control key (denoted `^`) to issue commands in nano.
- Unix systems don't like spaces in file names. Use `_` or `-` instead. Don't begin filenames with `.`, as this is interpreted as an option for most commands.
- On Unix systems, filename extensions can be important:
 - `.txt` for text
 - `.pdf` for pdf files
 - `.jpg` or `.png` for images
 - Usually not `.exe` for programs